

UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

ESCUELA DE INGENIERÍA DE SISTEMAS



Guía Práctica de Laboratorio

“MÁQUINA EXPENDEDORA”

“PROGRAMACIÓN II”

DOCENTE:

Ing. Breno Luque Zuñiga

ESTUDIANTE:

Arturo Sebastian Cutipa Flores

TACNA – PERÚ

2026



Guía de Laboratorio N° 02 Unidad 2 “MÁQUINA EXPENDEDORA# ”	3
I. INFORMACIÓN GENERAL	3
Objetivos	3
Equipos, materiales, programas y recursos utilizados	3
II. MARCO TEÓRICO	3
Máquina expendedora	3
Windows Forms	3
Programación Orientada a Objetos	3
DataGridView	4
III. PROCEDIMIENTO	4
Paso 1: Crear el proyecto	4
Paso 2: Diseñar la interfaz gráfica	4
Paso 3: Creación de clases	5
Paso 4: Declaración de listas	5
Paso 5: Declaración de la variable crédito	6
Paso 6: Registro de productos	6
Paso 7: Implementación de MostrarProductos()	6
Paso 8: Programación de botones de monedas	7
Paso 9: Programación de compra	7
El sistema:	7
● Buscaba el producto	7
● Verifica el crédito disponible	7
● Calculaba el vuelto	7
● Registraba la venta	7
● Mostraba mensajes de confirmación	7
Paso 10: Registro de ventas	7
Esto permitió llevar un control de todas las compras realizadas	8
Paso 11: Implementación de MostrarVentas()	8
El reporte se actualizaba automáticamente después de cada compra	8
Paso 12: Programación del botón Limpiar	8
También se limpiaba el TextBox del código del producto	8
Paso 13: Ejecución y pruebas	8
IV. ANÁLISIS E INTERPRETACIÓN DE RESULTADOS	9
Interfaz principal de la máquina expendedora	9
Registro de productos	9
Ingreso de monedas	10
Compra de producto	11
Cálculo de vuelto	12
Registro de ventas	12
Interpretación General	13
V. CUESTIONARIO	13
1. ¿Qué es una máquina expendedora?	13
2. ¿Qué función cumple Windows Forms?	13
3. ¿Qué es una clase en programación orientada a objetos?	14



4. ¿Qué es una lista genérica (List<T>)?	14
5. ¿Qué función cumple el DataGridView?	14
CONCLUSIONES	14
RECOMENDACIONES	14
BIBLIOGRAFÍA	14
WEBGRAFÍA	15



Guía de Laboratorio N° 02 Unidad 2 “MÁQUINA EXPENDEDORA#”

I. INFORMACIÓN GENERAL

Objetivos

- Desarrollar un juego interactivo tipo Snake utilizando Windows Forms.
- Aplicar programación orientada a objetos en C#.
- Implementar movimiento mediante teclado y temporizador.
- Utilizar gráficos para dibujar elementos del juego.
- Detectar colisiones y registrar puntaje.
- Reforzar conocimientos sobre eventos y lógica de programación.

Equipos, materiales, programas y recursos utilizados

- Computadora o laptop.
- Sistema operativo Windows.
- Visual Studio 2019/2022.
- Lenguaje de programación C#.
- Framework .NET Windows Forms.
- Editor de código integrado de Visual Studio.

II. MARCO TEÓRICO

Juego Snak

Snake es un videojuego clásico donde el jugador controla una serpiente que debe consumir alimentos para aumentar su tamaño sin chocar contra las paredes o su propio cuerpo.

Windows Forms

Windows Forms es una tecnología de .NET que permite desarrollar aplicaciones de escritorio con interfaz gráfica.

Facilita el uso de botones, paneles, temporizadores y eventos visuales.

Programación Orientada a Objetos

La programación orientada a objetos permite modelar entidades mediante clases y objetos.

En este proyecto se utilizaron las clases:

- SnakePart
- Food

Timer

- El componente Timer permite ejecutar instrucciones automáticamente en intervalos de tiempo definidos.
- En el juego se utilizó para mover continuamente la serpiente.

III. PROCEDIMIENTO

Paso 1: Crear el proyecto

Se abrió el programa Visual Studio y se creó un nuevo proyecto de tipo Windows Forms App (.NET Framework) con el nombre: Snake

Después de crear el proyecto, se ingresó al formulario principal (**Form1**) donde se desarrolló toda la aplicación.

Paso 2: Diseñar la interfaz gráfica

Desde el Toolbox se agregaron los controles necesarios para simular el funcionamiento de una máquina expendedora.

Control	Función
Panel	Área de juego
Label	Mostrar puntaje
Label	Mostrar Game Over
Button	Iniciar y reiniciar
Timer	Movimiento automático

Paso 3: Creación de la clase SnakePart

Se creó una clase llamada **SnakePart** para representar cada parte de la serpiente.

```
CSharp
public class SnakePart
```

```
{  
    public int X { get; set; }  
    public int Y { get; set; }  
}
```

La clase permite almacenar la posición de cada segmento de la serpiente.

Paso 4: Creación de la clase Food

Se creó una clase llamada **Food** para representar la comida del juego.

```
CSharp  
public class Food  
{  
    public int X { get; set; }  
    public int Y { get; set; }  
}
```

La clase permitió almacenar la posición de la comida dentro del tablero.

Paso 5: Declaración de variables

Dentro del formulario principal se declararon las variables necesarias para controlar el funcionamiento del juego

```
CSharp  
List<SnakePart> snake = new List<SnakePart>();  
  
Food comida = new Food();  
  
string direccion = "RIGHT";  
  
int puntos = 0;
```

Estas variables permitieron controlar la serpiente, dirección, comida y puntaje.

Paso 6: Programación del método IniciarJuego()

Se creó un método encargado de iniciar y reiniciar el juego.

```
CSharp
private void IniciarJuego()
{
    snake.Clear();

    snake.Add(new SnakePart()
    {
        X = 100,
        Y = 100
    });
}
```

Este método reiniciaba las variables y configuraba la posición inicial de la serpiente.

Paso 7: Generación de comida

Se implementó un método para generar comida aleatoriamente dentro del panel del juego.

```
CSharp
private void GenerarComida()
{
    comida.X = random.Next(0,
        panelJuego.Width / tamaño) * tamaño;
}
```

La comida aparecía automáticamente en diferentes posiciones del tablero

Paso 8: Programación del Timer

Se utilizó el componente Timer para mover automáticamente la serpiente

```
CSharp
private void timer1_Tick(object sender, EventArgs e)
{
    MoverSerpiente();

    VerificarColision();

    Comer();

    panelJuego.Invalidate();
}
```

El Timer ejecutaba continuamente la lógica principal del juego.

Paso 9: Movimiento de la serpiente

Se implementó un método encargado de mover la serpiente según la dirección seleccionada.

```
CSharp
if (direccion == "RIGHT")
    snake[0].X += tamaño;
```

El movimiento fue controlado mediante las teclas del teclado

Paso 10: Programación del teclado

Se utilizó el evento **KeyDown** para controlar la dirección de la serpiente.

```
CSharp
if (e.KeyCode == Keys.Right)
{
    direccion = "RIGHT";
}
```

Esto permitió mover la serpiente utilizando las flechas del teclado.

Paso 11: Detección de comida

Se implementó un método para detectar cuando la serpiente consumía la comida.

```
CSharp
if (snake[0].X == comida.X &&
    snake[0].Y == comida.Y)
{
    puntos++;
}
```

El reporte se actualizaba automáticamente después de cada compra.



V. CUESTIONARIO

1. ¿Qué es el juego Snake?

Snake es un videojuego clásico donde el jugador controla una serpiente que debe consumir alimentos para crecer sin colisionar con paredes o consigo misma.

2. ¿Qué función cumple el Timer en Windows Forms?

El Timer permite ejecutar instrucciones automáticamente en intervalos de tiempo definidos.

En el proyecto fue utilizado para mover continuamente la serpiente.

3. ¿Qué función cumple la clase Graphics?

La clase Graphics permite dibujar figuras y elementos visuales dentro de formularios o paneles.

En el juego se utilizó para dibujar la serpiente y la comida.

4. ¿Qué es una lista genérica (**List<T>**)?

Es una colección dinámica utilizada para almacenar objetos del mismo tipo.

En el proyecto se utilizó para almacenar las partes de la serpiente.

5. ¿Qué función cumple el evento **KeyDown**?

Permite detectar las teclas presionadas por el usuario.

En el juego fue utilizado para controlar la dirección de la serpiente.



CONCLUSIONES

- Se desarrolló correctamente un juego Snake utilizando Windows Forms y lenguaje C#.
- Se aplicaron conceptos de programación orientada a objetos mediante clases y listas genéricas.
- El Timer permitió controlar el movimiento automático de la serpiente.
- El uso de Graphics permitió dibujar los elementos gráficos del juego.
- El sistema detectó correctamente comida, colisiones y Game Over.
- El proyecto permitió reforzar conocimientos sobre eventos, gráficos y lógica de videojuegos.

RECOMENDACIONES

- Implementar almacenamiento en base de datos para guardar ventas permanentemente.
- Validar códigos incorrectos ingresados por el usuario.
- Incorporar más productos y categorías.
- Mejorar el diseño gráfico de la interfaz para simular una máquina real.

BIBLIOGRAFÍA

- Deitel, P. C#. Cómo programar. Pearson Educación.
- Microsoft Corporation. Manual de programación en C#.
- Joyanes Aguilar, L. Fundamentos de Programación.

WEBGRAFÍA

- Microsoft Learn. Documentación oficial de C#.
- Microsoft Learn. Guía de LINQ en .NET.
- Microsoft Learn. Windows Forms Documentation.